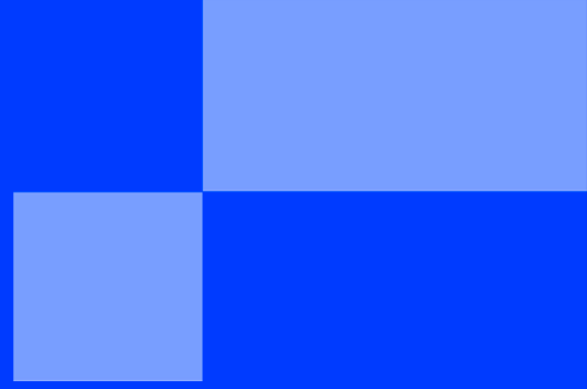




ATELIER STARTER - INTRODUCTION À L'ANALYSE DE DONNÉES

< WINTER CAMP 2026 />

ATELIER STARTER - INTRODUCTION À L'ANALYSE DE DONNÉES

Bienvenue dans ce premier atelier ! Notre mission est de répondre à la question : **Les pays obtiennent-ils plus de médailles quand ils organisent les Jeux Olympiques d'hiver ?**

Pour cela, nous allons explorer ensemble un jeu de données sur les médailles des JO d'hiver avec Python et la bibliothèque Pandas.

Étape 1 : Se familiariser avec l'environnement

Explication

Nous allons commencer par prendre en main l'environnement de travail, le **Jupyter Notebook**. C'est un outil interactif qui nous permet de mélanger du texte, comme celui-ci, et des blocs de code que l'on peut exécuter. Nous allons voir comment créer une cellule de texte (appelée **Markdown**) et une cellule de code.

Rendez-vous sur [Jupyter.org](https://jupyter.org)

- ✓ **Créer un nouveau notebook** : *New* → *Notebook* puis sélectionner le kernel **Python (Pyodide)**
- ✓ **Uploader le fichier CSV** : *Upload* → sélectionner `winter_olympics_medals.csv` → valider. Ou faire un drag'n drop de votre fichier CSV dans votre environnement Jupyter.

Expérimentation

- ✓ **Bloc de Markdown** :
Le Markdown est un langage de balisage simple qui permet de mettre en forme du texte facilement. Ajoutez un bloc de Markdown à votre notebook.

```
# Les pays obtiennent-ils plus de médailles quand ils organisent les JO d'hiver ?
```

La combinaison `Ctrl+Entrée` vous permet d'afficher le rendu de votre Markdown.

- ✓ **Bloc de code (Python)** :
Entrez votre premier bloc de code Python dans votre notebook.

```
print("Hello, world!")  
21 * 2
```

La combinaison **Ctrl+Entrée** vous permet d'exécuter votre bloc de code.



- ✓ Une ligne commençant par # en Markdown permet de déclarer un titre.
- ✓ Le résultat de la dernière expression entrée dans un bloc de code (ici `21 * 2`) de votre notebook est automatiquement affiché (si possible) après le bloc de code.

Étape 2 : Charger nos premières données

Explication

Nos données sont stockées dans un fichier au format **CSV**, un format texte simple pour les données en tableau. Pour les manipuler, nous allons utiliser une bibliothèque très puissante nommée **Pandas**. Elle nous permet de charger ces données dans un objet que l'on appelle un **DataFrame**, qui est l'équivalent d'un tableau intelligent.

Expérimentation :

Dans un bloc de code:

- importez la bibliothèque **Pandas**
- utilisez **Pandas** pour lire le fichier CSV dans un **DataFrame** (df)

```
import pandas

df = pandas.read_csv("winter_olympics_medals.csv")
```

Prenez le temps d'observer les données présentes dans le DataFrame:

- Le nombre de lignes
- Le nom des colonnes et les données qu'elles contiennent
- La colonne `host` qui indique si le pays a organisé les Jeux Olympiques cette année-là

Étape 3 : Isoler des informations (Filtrer)

Explication

Souvent, seule une partie de notre jeu de données nous intéresse. Nous allons donc apprendre à **filtrer** le DataFrame pour ne garder que les lignes qui correspondent à un critère précis.

Exemple

Par exemple, dans un nouveau bloc de code: pour extraire uniquement les médailles d'or de notre jeu de données:

```
df_medailles_or = df[df.medal == "gold"]
```

Expérimentation

- ✓ **Bloc de code (Python) :**
Essayez d'extraire toutes les médailles obtenues par la France dans notre jeu de données.
- ✓ **Bloc de Markdown :**
Rédigez une brève description des données que vous venez de filtrer.



- ✓ Utilisez **Ctrl+Entrée** pour exécuter chaque bloc (code ou Markdown) et afficher le résultat ou le rendu du texte.

Étape 4 : Résumer les données (Regrouper)

Explication

Une opération très courante est de **regrouper** les données pour en faire un résumé. Nous pouvons, par exemple, regrouper toutes les médailles par année pour savoir combien ont été décernées à chaque édition. Pour cela, nous utiliserons la fonction `groupby()`.

Exemple

Par exemple pour regrouper les médailles par année d'obtention:

```
df.groupby('year')
```

Il est ensuite possible d'appliquer un calcul sur chacun des groupes: ici on récupère leur taille.

```
df.groupby('year').size()
```

Expérimentation:

✓ Bloc de code:

Regroupez les médailles obtenues par la France par année. Comptez le nombre de médailles obtenues chaque année. Votre nouvelle colonne devrait s'appeler `medal_count`.

✓ Bloc de Markdown :

Rédigez une courte réponse à la question suivante:

- Quel est le nombre de médailles remportées par la France en 1968 et en 1992 ?



- ✓ `.reset_index(name="xxx")` permet de réinitialiser l'index d'une série/groupement en DataFrame tout en donnant le nom "xxx" à la nouvelle colonne contenant les valeurs.

Étape 5 : Rendre les chiffres visuels (Graphiques)

Explication

Un graphique est souvent plus facile à interpréter qu'un tableau. Nous allons utiliser une autre bibliothèque très populaire, **Matplotlib**, pour créer notre premier diagramme en barres.

Exemple

Commencez par importer **matplotlib** dans votre notebook.

```
import matplotlib.pyplot as plt
```

Par exemple, pour visualiser le nombre de médailles obtenues par la France par année (étape précédente):

```
df_medailles_france = df[df.country == "FRA"]

df_france_par_annee = df_medailles_france.groupby('year').size().reset_index(name="
    medals_count")

print(df_france_par_annee)

plt.bar(df_france_par_annee.year.astype(str), df_france_par_annee.medals_count)
plt.xticks(rotation=90)
plt.xlabel('Année')
plt.ylabel('Nombre de médailles')
plt.title('Nombre de médailles par année')
plt.show()
```

Expérimentation

✓ Bloc de code (Python) :

- Ajoutez le code nécessaire pour afficher un graphe (code donné ci-dessus) montrant le nombre de médailles obtenues par la France à chaque JO d'hiver.
- Vous pouvez ensuite tenter de personnaliser votre graphe.

✓ Explications :

Pouvez-vous expliquer chaque ligne de code de l'exemple précédent ?



- ✓ Un graphique peut être personnalisé avec les fonctions de matplotlib. Le type du graphique peut être modifié en utilisant différentes fonctions (`plt.plot()`, `plt.bar()`, `plt.barh()`, `plt.scatter()`).
- ✓ La couleur peut être spécifiée avec le paramètre `color` ('r', 'g', 'b', '#xyyyzz').

Étape 6 : Établir un classement (Trier)

Explication

Pour savoir quels pays ont obtenu le plus de médailles, nous devons trier notre tableau de données. En Pandas, nous utilisons pour cela la méthode `sort_values()`, qui permet d'organiser le DataFrame selon les

valeurs d'une ou plusieurs colonnes.

Expérimentation

✓ Bloc de code (Python) :

- Triez les pays par nombre de médailles obtenues
- Affichez le graphique correspondant



- ✓ Le paramètre `by` de `sort_values()` indique la colonne à utiliser pour le tri.
- ✓ Le paramètre `ascending=False` permet de trier par ordre décroissant.

Étape 7 : Se concentrer sur le podium (Top 5)

Explication

Un graphique avec trop de barres peut devenir illisible. Si notre tableau est déjà trié, nous pouvons facilement ne sélectionner que les N premières lignes pour nous concentrer sur le "Top 5" avec la méthode `.head(N)`.

Exemple

```
# Après avoir utilisé .head(3), vous aurez un DataFrame  
# contenant uniquement les 3 premières lignes du tableau.  
classement_artistes.head(3)
```

Expérimentation

- ✓ Dans un nouveau bloc de code :
- ✓ Utilisez simplement `.head(5)` sur votre DataFrame déjà trié de l'étape précédente pour obtenir le top 5.
- ✓ Affichez ensuite le graphique correspondant (par exemple un barplot avec matplotlib ou pandas plotting) sur ce top 5 uniquement.

v1

{EPITECH}